

VoxlyOne

Plataforma de Aprendizizado de Inglês — MVP

Documento de Arquitetura Técnica & Requisitos

Versão	2.5.0 (pivô do VOXLY v1.1 + 5 rodadas de revisão)
Data	Junho 2026
Stack	PHP 8.x puro + MySQL + JS Vanilla
Ambiente	Hostinger — Hospedagem Compartilhada
IDE	VS Code + Claude Code
Domínio	voxly.hubfy.app

1. VISÃO GERAL E DECISÃO DE ARQUITETURA

1. Visão Geral e Decisão de Arquitetura

1.1 O Produto

O VoxlyOne é a versão MVP da plataforma Voxly: um app web para brasileiros treinarem pronúncia em inglês. O usuário cadastra frases (PT/EN), ouve a pronúncia correta em três velocidades, grava a própria voz, recebe análise de IA com nota de 0 a 10 e avança com nota ≥ 8.0 (a nota 10 é o selo 'Dominada'). Frases dominadas ficam numa lista de conquistas com replay.

1.2 Por que o pivô para PHP puro

Simplicidade	Sem build, sem Node, sem VPS: deploy é subir arquivos no File Manager da Hostinger
Custo	Hospedagem compartilhada já contratada — zero custo adicional de infraestrutura
Velocidade de MVP	Menos camadas = menos pontos de falha e menos setup — foco total no produto
Realidade da carga	O trabalho pesado (gravação, TTS) roda no browser; o PHP só autentica, persiste e repassa o áudio à OpenAI
Migração futura	Código organizado permite migrar para a VPS (que o dono já possui) sem reescrever — apenas mover arquivos

 Limite honesto desta arquitetura: hospedagem compartilhada atende dezenas de usuários simultâneos com folga (perfeito para MVP), mas não centenas — cada análise de IA ocupa um processo PHP por 8–15s. Quando o app crescer, migrar para a VPS Hostinger sem mudança de código.

1.3 Decisões técnicas chave do MVP

- Gravação com MediaRecorder nativo (webm/opus no Chrome-Firefox, mp4/aac no Safari) + conversão ÚNICA ao final para WAV 16kHz mono via OfflineAudioContext — elimina ffmpeg no servidor e evita a armadilha do iOS, que roda o AudioContext a 48kHz e ignora o sample rate pedido
- GPT-4o Audio (gpt-4o-audio-preview) via cURL — avalia o áudio bruto da pronúncia real (nunca Whisper, que 'corrige' o sotaque na transcrição)
- Google OAuth 2.0 implementado manualmente em PHP (sem SDK pesado) — fluxo authorization code com endpoints oficiais do Google
- Sessões PHP nativas com cookie httponly + secure para autenticação pós-login
- Interface mobile-first: o público principal usa celular

2. REQUISITOS FUNCIONAIS

2. Requisitos Funcionais

2.1 Autenticação

RF-01 — Login com Google OAuth 2.0 (PHP puro)

Fluxo	Authorization Code Flow: login.php redireciona para accounts.google.com → callback.php troca o code por tokens → busca perfil em openidconnect.googleapis.com/v1/userinfo → upsert do usuário (INSERT ... ON DUPLICATE KEY UPDATE name, avatar_url — email e google_id são ambos UNIQUE; um INSERT simples daria erro 500 se o mesmo e-mail voltasse com outro google_id) → inicia sessão PHP → redirect para dashboard.php
Credenciais	GOOGLE_CLIENT_ID e GOOGLE_CLIENT_SECRET no config.php (fora do public_html)
Sessão	session_start() com cookie httponly, secure e samesite=Lax. Regenerar session_id no login (session_regenerate_id).
State	Parâmetro state aleatório (random_bytes) validado no callback contra CSRF
Logout	logout.php: session_destroy() + limpar cookie + redirect para index.php
Dados salvos	google_id, email, name, avatar_url, created_at (coluna plan só na Fase 2, quando premium existir)

RF-02 — Proteção de páginas

- Todas as páginas exceto index.php (landing/login), privacy.php e terms.php exigem sessão ativa
- Helper require_auth() no topo de cada página protegida: sem sessão → redirect para index.php
- Endpoints de API (pasta /api) retornam 401 JSON se sem sessão — nunca redirect

2.2 Gerenciamento de Frases

RF-03 — Cadastro de Frases

Campos	text_en OBRIGATÓRIO (é o que se pratica; se o usuário digitar só PT, traduzir ANTES de salvar); text_pt opcional (NULL); máx 200 caracteres cada; category (existente ou nova); level (beginner/intermediate/advanced); phonetic_guide (opcional)
Tradução automática	Se apenas um idioma preenchido, botão 'Traduzir' chama api/translate.php (GPT-4o mini) e preenche o outro campo. No MESMO JSON, o endpoint retorna phonetic_br — pronúncia aproximada em sons do português, sem IPA (ex: 'How are you?' → 'RRAU ar iú') — pré-preenchendo o phonetic_guide (editável). Custo marginal: alguns tokens de saída. Botão 'Gerar fonética' SEMPRE disponível — mesmo

	com PT e EN já preenchidos (o caso mais comum de quem cadastra os dois), chamando o mesmo endpoint
Validação	Server-side em PHP (nunca confiar só no JS): trim, campos obrigatórios, mb_strlen ≤ 200 (text_pt/text_en) e ≤ 400 (phonetic_guide — pré-preenchido pela IA via phonetic_br, mesma classe de bug do heard: sem o limite, estouraria o INSERT no strict mode)

RF-04 — Listagem e Organização

- Listagem paginada (20/página) com filtros por categoria, nível e status via query string
- Toggle PT/EN em cada card (JS puro, sem reload)
- Status visual: Não iniciada / Em progresso / Aprovada / Dominada — 'Aprovada' (best_score ≥ 8 e < 10) é rótulo DERIVADO do best_score na exibição; o ENUM do schema não muda
- Ordenação: por data de criação, por nível ou por status
- Busca por texto com LIKE em text_pt e text_en — suficiente para dezenas de frases por usuário; sem FULLTEXT no MVP

RF-05 — Edição e Exclusão

- Edição de qualquer campo da frase própria; exclusão com confirmação (modal JS)
- DELETE em cascata remove tentativas da frase (FK ON DELETE CASCADE)
- Toda query filtra por user_id da sessão — usuário nunca vê dados de outro

2.3 Player de Áudio

RF-06 — Reprodução em Inglês (Web Speech API)

Tecnologia	speechSynthesis do browser — gratuito, sem custo de API
Velocidades	Slow: rate 0.6 Normal: rate 0.9 Fast: rate 1.2 — três botões, seleção persiste na sessão (sessionStorage)
Voz	Selecionar a melhor voz en-US disponível via getVoices() (preferir 'Google US English' no Chrome, 'Samantha' no Safari)
Regra	GRAVAR habilita no evento onstart do play + timer de fallback (nº de palavras × 400ms, mínimo 2s); onend é tratado como bônus. NUNCA depender só do onend: no Safari iOS ele frequentemente não dispara — travaria o app exatamente no público mobile que é o alvo

 Qualidade da voz varia por browser (boa no Chrome/Edge e Safari, robótica no Firefox). Upgrade futuro: OpenAI TTS com cache em MP3 no servidor — frases repetidas custam zero.

2.4 Gravação de Voz

RF-07 — Captura em WAV no Browser

Tecnologia	MediaRecorder nativo para capturar (robusto em todos os browsers) + conversão única ao FINAL: decodeAudioData → OfflineAudioContext reamostra para WAV 16kHz mono com pitch correto
Por que assim	Elimina ffmpeg no servidor E a armadilha do iOS (AudioContext fixo em 48kHz — gravar direto com header '16000' entregaria áudio acelerado à IA). ScriptProcessorNode é depreciado e engasga a UI em celular fraco.
Onda animada	AnalyserNode ligado ao stream ao vivo — separado da gravação, custo mínimo
Tamanho	16kHz mono 16-bit ≈ 32KB/segundo — frase de 10s ≈ 320KB (aceitável para upload)
Duração	Mínimo 1s, máximo 30s (corte automático)
UX	Permissão de microfone com mensagem clara se negada; animação de onda + contador durante gravação; preview antes de analisar; botão regravar
HTTPS	Obrigatório (SSL Let's Encrypt já ativo no domínio) — sem HTTPS o getUserMedia não funciona

```
// Conversão única ao final da gravação (resolve o iOS a 48kHz):
const ctx = new AudioContext();
const decoded = await ctx.decodeAudioData(await blob.arrayBuffer());
const off = new OfflineAudioContext(1,
  Math.ceil(decoded.duration * 16000), 16000);
const src = off.createBufferSource();
src.buffer = decoded; src.connect(off.destination); src.start();
const pcm = await off.startRendering(); // 16kHz mono, reamostrado correto
// encodeWav(pcm): header RIFF de 44 bytes + Int16Array
await ctx.close(); // liberar - contextos acumulados travam a próxima gravação em
celular fraco
```

2.5 Análise de IA e Feedback

RF-08 — Pipeline GPT-4o Audio (uma etapa)

O WAV gravado é enviado via POST multipart para api/analyze.php, que codifica em base64 e chama o GPT-4o Audio via cURL. O modelo ouve o áudio real e retorna JSON com nota e feedback.

 **NUNCA** usar Whisper para transcrever antes de avaliar: o Whisper corrige o sotaque na transcrição e mascara os erros de pronúncia. O GPT-4o Audio avalia o áudio bruto.

- temperature: 0 reduz a variação, mas modelos de áudio NÃO garantem determinismo — a mesma gravação pode oscilar ~1 ponto entre execuções (mais um motivo para o avanço ser ≥ 8.0, e não um gate exato em 10)
- set_time_limit(60) no início de analyze.php + CURLOPT_CONNECTTIMEOUT 10 + CURLOPT_TIMEOUT 25 — o total deve ser MENOR que o teto do LiteSpeed: devolver 'tente novamente' em 25s é melhor que uma conexão morta em 45s
- Resposta do modelo validada com json_decode + verificação de campos antes de salvar

RF-09 — Estrutura do Feedback (JSON)

score	0.0 a 10.0 (uma casa decimal). 0 = incompreensível; 10 = fluente como nativo
--------------	--

heard	O que o modelo ouviu o usuário dizer
positives	O que acertou (sempre presente, mesmo em nota baixa) — PT-BR, máx 80 chars
errors[]	{ word, said, correct, tip } por palavra com problema
naturalness	Ritmo, entonação e fluidez — PT-BR, máx 80 chars
speed_feedback	Rápido/lento demais para o nível — PT-BR, máx 60 chars
main_tip	A dica mais importante e memorável — PT-BR, máx 100 chars

RF-10 — Regra de Progressão

- AVANÇAR ≠ DOMINAR: o usuário avança para a próxima frase com nota ≥ 8.0 ; a nota 10 é o selo 'Dominada' (confetti + página de conquistas). Um gate exato em 10 travaria o produto: o modelo raramente dá 10 a não-nativos e a nota oscila ~1 ponto entre execuções
- Cada tentativa salva em attempts (nota, heard, feedback JSON, timestamp)
- Sem limite de tentativas; a melhor nota é a exibida no card
- Anti-frustração: a partir da 8ª tentativa sem atingir 8.0, aparece 'Pular por agora' — atualiza last_attempt_at e a frase volta ao FIM da fila
- Fila da 'próxima frase': WHERE user_id=? AND status<>'mastered' ORDER BY (COALESCE(best_score,0) >= 8) ASC, last_attempt_at IS NULL DESC, last_attempt_at ASC, created_at ASC LIMIT 1 — SEM filtro por best_score: filtrar best_score<8 faria frases aprovadas (ex: 8.5, não-mastered) sumirem da fila para sempre e a fila esvaziaria no cenário mais provável. Na ordenação, aprovadas-não-dominadas vão para o FIM da fila. O filesort da expressão é irrelevante com dezenas de frases; idx_queue segue útil no WHERE
- Nota 10 → status 'mastered' + mastered_at + animação de celebração
- Status 'mastered' NUNCA regride: praticar uma frase já dominada registra a tentativa no histórico, mas não altera status, best_score para baixo, nem sobrescreve mastered_at
- best_score NUNCA regride em NENHUM status: UPDATE com best_score = GREATEST(COALESCE(best_score,0), :score) — sem isso, tirar 6.0 ao 'Tentar o 10' desfaria uma aprovação de 8.5 e devolveria a frase à fila principal
- Transições de status (completas — não existem outras): not_started até a primeira tentativa; primeira tentativa registrada → in_progress; score 10 → mastered

2.6 Lista de Conquistas

RF-11 — Frases Dominadas

- Página mastered.php: cards com EN/PT, categoria, nível, data da conquista, nº de tentativas
- Play com seletor de velocidade em cada card
- Contador total de dominadas no perfil

2.7 Privacidade e Conta

RF-12 — Consentimento e Exclusão

Consentimento	Modal antes da primeira gravação: áudio enviado para análise por IA e descartado em seguida. Aceite salvo com timestamp (consented_at no users).
----------------------	--

Páginas legais	privacy.php e terms.php públicas, linkadas no rodapé e na tela de login
Exclusão de conta	Botão no perfil: DELETE do usuário remove frases, tentativas e categorias em cascata
Retenção de áudio	WAV processado em memória (php://input / \$_FILES) e descartado — nunca gravado em disco
LGPD	Aplica-se mesmo operando dos EUA (Art. 3º — oferta de serviço a residentes no Brasil). Consentimento + exclusão + minimização cobrem o essencial do MVP.

3. REQUISITOS NÃO FUNCIONAIS

3. Requisitos Não Funcionais

3.1 Performance e Limites da Hospedagem Compartilhada

Operação	Meta	Máx aceitável	Observação
Carregamento de página	< 1.5s	< 3s	PHP renderiza HTML direto; CSS/JS minificados
Play TTS	< 500ms	< 1s	Roda no browser
Upload WAV (10s)	< 2s	< 4s	~320KB
Análise IA	< 10s	< 20s	GPT-4o Audio; loading animado obrigatório
Listagem de frases	< 800ms	< 2s	Índices + paginação

- Hospedagem compartilhada Hostinger: ~20–40 processos PHP simultâneos (varia por plano) — suficiente para dezenas de usuários ativos ao mesmo tempo
- Análise de IA é o gargalo: cada uma segura um processo por 8–15s. Rate limit essencial (ver 3.2)
- php.ini via hPanel: max_execution_time 60, upload_max_filesize 8M, post_max_size 10M, memory_limit 256M
- Plano de escala: mover os mesmos arquivos para a VPS Hostinger quando necessário — zero reescrita

🚨 RISCO CRÍTICO A VALIDAR NO DIA 1: max_execution_time NÃO conta tempo de espera de I/O externo — os 8-15s da OpenAI não entram nessa conta. Quem mata o processo é o timeout de wall-clock do LiteSpeed da Hostinger, fora do controle no plano compartilhado. Se esse teto for menor que ~20s, a arquitetura não funciona. Validação obrigatória: subir api/_spike.php que faz UMA chamada real ao GPT-4o Audio e imprime o tempo decorrido, ANTES de escrever qualquer outra linha. Se falhar, há um plano B barato antes da VPS: stream: true na OpenAI repassando os chunks com flush() — proxies matam conexão por IDLE, não por duração total; com bytes fluindo o LiteSpeed geralmente não derruba. Só se ambos falharem, VPS.

3.2 Segurança

SQL Injection	PDO com prepared statements em TODAS as queries — nunca concatenar input em SQL
XSS	htmlspecialchars(x, ENT_QUOTES, 'UTF-8') em toda saída de dado do usuário
CSRF	Token por sessão em todos os forms POST + validação server-side; state no OAuth
Sessão	httponly + secure + samesite=Lax; session_regenerate_id no login

Secrets	config.php FORA do public_html (ex: /home/user/config/config.php) com OPENAI_API_KEY, GOOGLE_CLIENT_SECRET e credenciais MySQL
Rate limiting	Tabela rate_limits: máx 30 análises/dia e 50 traduções/dia por usuário + teto GLOBAL diário (botão de emergência: acima dele, responder 'sistema em manutenção'). Incremento ATÔMICO via INSERT...ON DUPLICATE KEY UPDATE antes da chamada — nunca check-depois-grava (duas abas furariam). Devolução da cota: em QUALQUER caminho que não grave um attempt (timeout, HTTP 429/500/503, saldo esgotado, abort pelo teto global) — try/finally até o commit(); a única exceção é erro de validação do usuário
Freio de custo REAL	Saldo pré-pago na OpenAI com auto-recarga DESLIGADA. O alerta de billing avisa; só o saldo impede. Rate limit é freio de abuso, não de custo: 30/dia × ~\$0.03 × 30 dias ≈ \$27/mês por usuário no limite — poucos usuários intensos estouram o orçamento de \$60/mês. Teto global: GLOBAL_DAILY_ANALYSES = 300 (≈\$9/dia no pior caso), a calibrar com o usage real medido no spike
Uploads	Aceitar apenas audio/wav até 2MB; validar magic bytes (RIFF); nunca salvar em disco
Erros	display_errors Off em produção; log em arquivo fora do public_html; mensagens genéricas ao usuário
Headers	.htaccess: X-Content-Type-Options, X-Frame-Options DENY, Referrer-Policy

3.3 Usabilidade (Mobile-First)

- Layout projetado primeiro para 360–430px (celulares) e expandido para desktop
- Botões de ação com mínimo 44×44px (padrão de toque)
- Feedback visual em toda ação: spinners, toasts, estados do botão de gravação
- Análise em andamento: indicador animado com mensagem ('Ouvindo sua pronúncia...')
- Nota com animação ao aparecer; confetti na nota 10
- Contraste AA (WCAG 2.1); labels em todos os inputs

3.4 Confiabilidade e Tratamento de Erros

Microfone negado	Instrução visual de como liberar no browser/celular
Falha na análise	Erro amigável + 'Tentar novamente' — a gravação permanece no browser, sem regravar
Timeout OpenAI	CURLOPT_TIMEOUT 25s; se estourar, erro tratado, cota devolvida (count-1) e gravação preservada no browser
Offline	navigator.onLine + captura de fetch failure com mensagem clara
Resposta inválida da IA	json_decode falhou ou campos ausentes → tratar como erro, não salvar tentativa

3.5 Manutenibilidade e Observabilidade

- PHP 8.2+ com `declare(strict_types=1)` em todos os arquivos
- Estrutura organizada (ver seção 5) — separação clara entre páginas, API, includes e assets
- Funções e variáveis em inglês; comentários em português
- Log de erros com `error_log()` para arquivo dedicado; revisar semanalmente
- `api/health.php` público: executa `SELECT 1` no MySQL e retorna `{ status: 'ok' }` — monitorar com UptimeRobot gratuito
- Backup: export semanal do MySQL via hPanel (ou cron com `mysqldump`) + download mensal local
- Alerta de billing na OpenAI em \$50/mês

4. BANCO DE DADOS (MySQL)

4. Banco de Dados (MySQL 8 / MariaDB)

Engine InnoDB, charset utf8mb4, collation utf8mb4_unicode_ci. Criar via phpMyAdmin no hPanel.

4.1 Schema Completo

```
CREATE TABLE users (
  id          INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  google_id   VARCHAR(64)  NOT NULL UNIQUE,
  email       VARCHAR(255) NOT NULL UNIQUE,
  name        VARCHAR(120) NOT NULL,
  avatar_url  VARCHAR(500),
  -- plan (free/premium): adicionar na Fase 2, quando premium existir
  consented_at DATETIME NULL,          -- consentimento de gravação (RF-12)
  created_at   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE categories (
  id          INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id     INT UNSIGNED NOT NULL,
  name        VARCHAR(80) NOT NULL,
  created_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uq_user_cat (user_id, name),
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE phrases (
  id          INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id     INT UNSIGNED NOT NULL,
  category_id INT UNSIGNED NULL,
  text_pt     VARCHAR(200) NULL,          -- opcional; text_en é o obrigatório
  text_en     VARCHAR(200) NOT NULL,
  phonetic_guide VARCHAR(400) NULL,      -- fonética PT-BR é mais longa que o
original
  level       ENUM('beginner','intermediate','advanced') NOT NULL,
  status      ENUM('not_started','in_progress','mastered')
              NOT NULL DEFAULT 'not_started',
  best_score  DECIMAL(3,1) NULL,          -- 0.0 a 10.0
  attempts_count INT UNSIGNED NOT NULL DEFAULT 0,
  mastered_at DATETIME NULL,
  last_attempt_at DATETIME NULL,          -- ordenação da fila / skip
  created_at   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  KEY idx_user_status (user_id, status),
  KEY idx_queue (user_id, status, last_attempt_at),
  KEY idx_category (category_id),
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  FOREIGN KEY (category_id) REFERENCES categories(id) ON DELETE SET NULL
)
```

```

) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE attempts (
  id          INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  phrase_id   INT UNSIGNED NOT NULL,
  user_id     INT UNSIGNED NOT NULL,
  score       DECIMAL(3,1) NOT NULL,
  heard       VARCHAR(500) NULL,          -- o que a IA ouviu (modelo pode ser
verboso)
  feedback_json JSON NOT NULL,
  audio_seconds TINYINT UNSIGNED NULL,    -- derivado: (bytes - 44) / 32000
  created_at   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  -- Áudio NÃO é armazenado no MVP (descartado após análise)
  KEY idx_phrase (phrase_id),
  KEY idx_user (user_id),
  FOREIGN KEY (phrase_id) REFERENCES phrases(id) ON DELETE CASCADE,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

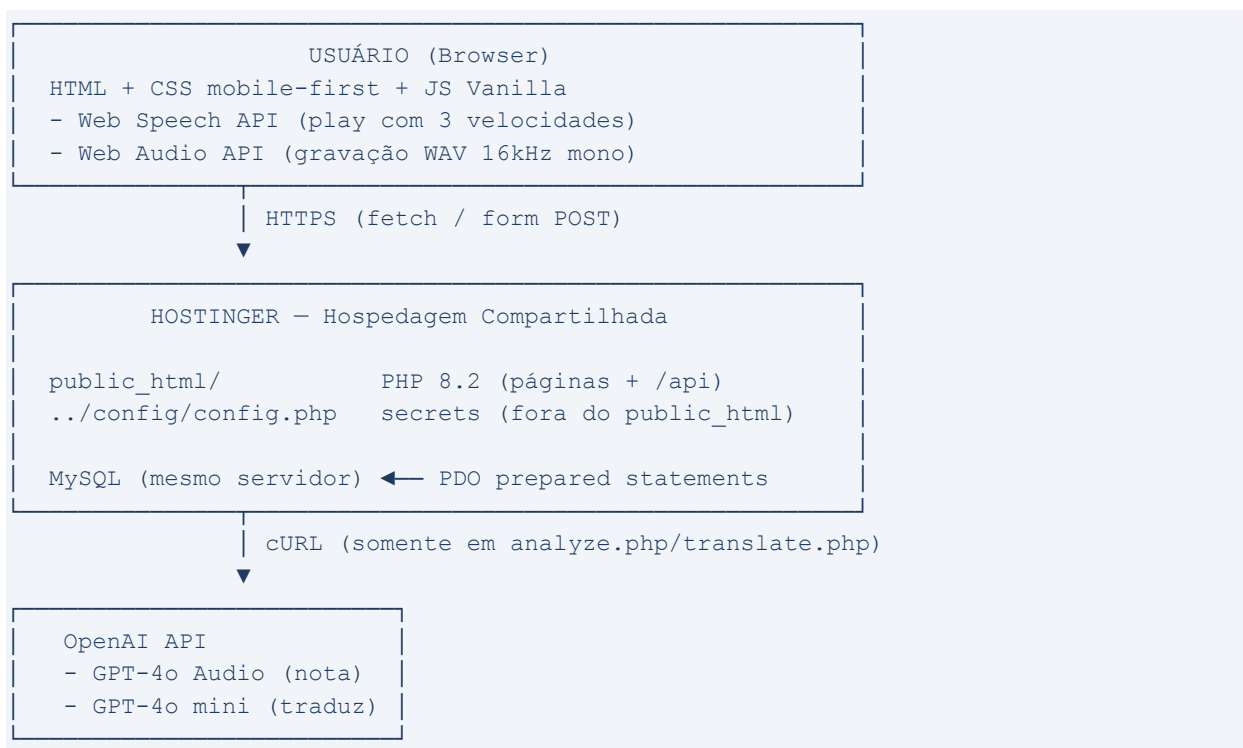
CREATE TABLE rate_limits (
  user_id     INT UNSIGNED NOT NULL,
  action      ENUM('analyze','translate') NOT NULL,
  day         DATE NOT NULL,
  count       INT UNSIGNED NOT NULL DEFAULT 0,
  PRIMARY KEY (user_id, action, day),
  KEY idx_action_day (action, day),      -- teto global sem full scan
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

5. ARQUITETURA E ESTRUTURA DE ARQUIVOS

5. Arquitetura e Estrutura de Arquivos

5.1 Diagrama



5.2 Estrutura de Arquivos

```
/home/USUARIO/
├── config/
│   └── config.php          # OPENAI_API_KEY, GOOGLE_CLIENT_ID/SECRET,
│                           # credenciais MySQL – FORA do public_html
├── logs/
│   └── app.log             # error_log() do app
├── includes/
│   ├── bootstrap.php      # FORA do public_html – inacessível via web
│   ├── auth.php           # require config, session, PDO, helpers
│   ├── db.php             # require_auth(), current_user()
│   ├── csrf.php           # conexão PDO singleton
│   ├── rate_limit.php     # geração/validação de token
│   ├── openai.php         # incremento atômico + teto global
│   ├── header.php         # call_gpt4o_audio(), call_gpt4o_mini()
│   └── footer.php         # <head> + nav (mobile-first)
└── public_html/
```

```

├── index.php          # landing + botão 'Entrar com Google'
├── auth/
│   ├── login.php     # monta URL do Google e redireciona (com state)
│   ├── callback.php  # troca code por token, cria sessão
│   └── logout.php
├── dashboard.php     # lista de frases (filtros, busca, paginação)
├── phrase_form.php    # criar/editar frase
├── practice.php      # tela de prática (play, gravar, feedback)
├── mastered.php      # conquistas
├── profile.php       # perfil, estatísticas, excluir conta
├── privacy.php       # política de privacidade (pública)
├── terms.php         # termos de uso (pública)
├── api/
│   ├── analyze.php   # POST WAV → GPT-4o Audio → JSON
│   ├── translate.php # POST texto → GPT-4o mini → JSON
│   ├── phrases.php   # CRUD via POST + action=create|update|delete
│   ├── consent.php   # grava consented_at (RF-12)
│   ├── delete_account.php
│   └── health.php    # SELECT 1 no MySQL + { status: 'ok' }
├── assets/
│   ├── css/app.css   # mobile-first, variáveis CSS
│   └── js/
│       ├── recorder.js # MediaRecorder + OfflineAudioContext → WAV 16kHz
│       ├── player.js   # speechSynthesis + 3 velocidades
│       └── practice.js # orquestra a tela de prática
└── .htaccess         # headers de segurança + bloqueios

```

6. ENDPOINTS E INTEGRAÇÃO OPENAI

6. Endpoints e Integração OpenAI

6.1 POST /api/analyze.php

Auth	Sessão PHP ativa (401 JSON caso contrário)
Content-Type	multipart/form-data
Campos	audio (WAV ≤ 2MB), phrase_id, csrf_token
Validações	Sessão + consented_at preenchido (403 se não — modal só no client-side seria decorativo); CSRF válido; frase pertence ao user; magic bytes RIFF; rate limit 30/dia (incremento atômico) + teto global diário
Fluxo	set_time_limit(60) → valida → incrementa rate limit (atômico; aborta se acima do teto) → base64 do WAV → cURL GPT-4o Audio (connect 10s, total 25s) → json_decode + validação de campos → sanitização: mb_substr(heard, 0, 500) e clamp do score em 0–10 com cast float (o modelo pode devolver 11, '9.5' ou negativo; um heard de 600 chars estouraria o INSERT no strict mode DEPOIS de pagar a chamada) → audio_seconds derivado no servidor: (bytes - 44) / 32000 (WAV 16kHz mono 16-bit — de graça, sem campo no POST) → beginTransaction() → INSERT attempt + UPDATE phrase com best_score = GREATEST(COALESCE(best_score,0), :score), status, counts e last_attempt_at = NOW() em TODA tentativa (não só no skip: é o critério de ordenação da fila — sem isso, frase tentada 7× continuaria NULL e nunca cederia o topo às frases novas) → commit() → resposta JSON. Sem transação, uma falha entre os dois deixa attempts_count divergente para sempre.
Resposta 200	{ success:true, data:{ attempt_id, score, feedback, is_mastered } }
Erros	{ success:false, error:{ code, message } } — mensagens genéricas, log completo no servidor

```
-- Incremento atômico do rate limit (ANTES da chamada à OpenAI):
INSERT INTO rate_limits (user_id, action, day, count)
VALUES (?, 'analyze', CURDATE(), 1)
ON DUPLICATE KEY UPDATE count = count + 1;
-- SELECT count em seguida; se > 30, abortar (limite do usuário).

-- Teto global diário (botão de emergência):
SELECT COALESCE(SUM(count),0) FROM rate_limits
WHERE action='analyze' AND day=CURDATE();
-- Acima do teto global → 'sistema em manutenção, tente mais tarde'.

-- REGRA GERAL DE DEVOLUÇÃO: devolver a cota (count = count - 1) em
-- QUALQUER caminho que não grave um attempt — try/finally: se não
-- chegou ao commit(), decrementa. Cobre timeout, HTTP 429/500/503,
-- insufficient_quota (saldo esgotado) e o abort pelo teto global
```

```
-- (o incremento vem ANTES da checagem do SUM – sem devolução, o
-- usuário perderia cota por um limite que não é culpa dele).
-- Única exceção: erro de VALIDAÇÃO do usuário – aí não devolve.
```

6.2 Chamada cURL — GPT-4o Audio

```
$payload = [
    'model' => OPENAI_AUDIO_MODEL, // definido em config.php
    'temperature' => 0,
    'modalities' => ['text'],
    'messages' => [
        ['role' => 'system', 'content' => SYSTEM_PROMPT],
        ['role' => 'user', 'content' => [
            ['type' => 'text', 'text' => $userPrompt],
            ['type' => 'input_audio',
                'input_audio' => ['data' => $audioBase64, 'format' => 'wav']],
        ]],
    ],
];

$ch = curl_init('https://api.openai.com/v1/chat/completions');
curl_setopt_array($ch, [
    CURLOPT_RETURNTRANSFER => true,
    CURLOPT_POST => true,
    CURLOPT_CONNECTTIMEOUT => 10,
    CURLOPT_TIMEOUT => 25, // menor que o teto do LiteSpeed
    CURLOPT_HTTPHEADER => [
        'Content-Type: application/json',
        'Authorization: Bearer ' . OPENAI_API_KEY,
    ],
    CURLOPT_POSTFIELDS => json_encode($payload),
]);
```

6.3 Prompts

```
// SYSTEM_PROMPT:
'You are an English pronunciation coach specialized in Brazilian
Portuguese speakers. You will receive an audio recording of a student
attempting to say a target phrase. Listen carefully to the ACTUAL
pronunciation, rhythm, intonation and speed.
Always respond with ONLY a valid JSON object, no markdown.'

// USER_PROMPT (montado em PHP)
// SEM attempt_number: informar que é a 9ª tentativa envia a nota para
// cima (modelos 'recompensam' persistência) – variação gratuita a eliminar.
Target phrase: "{text_en}"
Level: {level}

Listen to the attached audio and evaluate the pronunciation.
Return ONLY this JSON:
{
```

```

"score": number (0-10, one decimal),
"heard": string (what the student actually said),
"positives": string (PT-BR, max 80 chars),
"errors": [{ "word", "said", "correct", "tip" }],
"naturalness": string (PT-BR, max 80 chars),
"speed_feedback": string (PT-BR, max 60 chars),
"main_tip": string (PT-BR, max 100 chars)
}

```

Scoring: 0-2 incomprehensible | 3-4 very hard to understand |
 5-6 understandable with significant errors | 7-8 good, minor errors |
 9 near-native | 10 native-level fluency and naturalness

6.4 Demais Endpoints

Endpoint	Método/Auth	Função
api/translate.php	POST / sessão + CSRF	Traduz PT↔EN com GPT-4o mini e retorna também phonetic_br no mesmo JSON (pronúncia aproximada em sons do PT-BR, sem IPA); rate limit 50/dia (20/dia apertaria cadastro em lote)
api/phrases.php	POST / sessão + CSRF	CRUD via campo action=create update delete — só POST: PUT/DELETE em hospedagem compartilhada dão dor de cabeça (corpo não chega em \$_POST, WAFs bloqueiam)
api/consent.php	POST / sessão + CSRF	UPDATE users SET consented_at = NOW() WHERE id = ? — sem este endpoint, o 403 do analyze.php travaria a primeira análise de TODO usuário (o modal aceitaria no browser e o valor nunca chegaria ao banco)
api/delete_account.php	POST / sessão + CSRF	Exclui usuário e dados em cascata; destrói sessão
api/health.php	GET / público	Executa SELECT 1 no MySQL e retorna { status:'ok' } — sem tocar o banco, reportaria ok com o MySQL fora do ar

7. FLUXO DA TELA DE PRÁTICA

7. Fluxo da Tela de Prática (practice.php)

1. Página carrega com a frase (PT + EN + pronúncia aproximada) e seletor de velocidade
2. Usuário escolhe Slow / Normal / Fast
3. PLAY → speechSynthesis fala a frase em en-US
4. GRAVAR habilita no onstart do play + timer de fallback (iOS: onend não confiável)
5. GRAVAR → Web Audio API captura WAV (onda animada + contador)
6. PARAR → preview do áudio + botões [Regravar] [Analisar]
7. ANALISAR → fetch POST para api/analyze.php
loading: 'Ouvindo sua pronúncia...' (8-15s)
8. FeedbackCard renderiza: nota animada, heard, erros, dica
 - └ nota < 8.0 → [Tentar novamente] (volta ao passo 2)
 - └ 8ª tentativa sem 8.0 → aparece [Pular por agora]
 - └ 8.0 ≤ nota < 10 → 'Aprovada!' + [Próxima frase] + [Tentar o 10]
 - └ nota = 10 → confetti + 'Dominada!' + [Próxima frase]

7.1 Páginas do App

Página	Arquivo	Descrição
Login	index.php	Landing com botão Google, links privacy/terms
Dashboard	dashboard.php	Lista paginada, filtros, busca, FAB de nova frase
Nova/Editar Frase	phrase_form.php	Form com tradução automática e guia fonético
Prática	practice.php	Coração do app — fluxo acima
Conquistas	mastered.php	Cards de dominadas com play
Perfil	profile.php	Estatísticas, excluir conta

8. SETUP E DEPLOY

8. Setup e Deploy

8.1 Ordem de Desenvolvimento

1. DIA 1 — SPIKE DE VALIDAÇÃO (obrigatório, ~15 min): `api/_spike.php` faz UMA chamada real ao GPT-4o Audio com WAV de exemplo e imprime: (a) o tempo decorrido — valida o teto do LiteSpeed; (b) o campo `usage` da resposta (tokens de áudio + saída) — calibra o custo REAL por análise em vez de chutar \$0.03; (c) o resultado do `gpt-4o-mini-audio-preview` com a MESMA gravação, lado a lado — se a nota for coerente, o custo do MVP cai uma ordem de grandeza trocando uma linha do config. SEGURANÇA: proteger com token na query string (sem token → 404) e DELETAR o arquivo ao fim do Dia 1 — exposto no `public_html`, ele carrega sua chave OpenAI e qualquer varredura queima o saldo, que é seu único freio real de custo. Se o processo for morto: testar o plano B do streaming; só então, VPS
2. Google Cloud Console: criar OAuth Client ID (tipo Web), origin `https://voxly.hubfy.app`, redirect `https://voxly.hubfy.app/auth/callback.php`
3. MySQL: criar banco e usuário no hPanel; rodar o schema da seção 4 no phpMyAdmin
4. `config.php` fora do `public_html` + `includes/bootstrap.php`
5. Autenticação completa: `login.php`, `callback.php`, `logout.php`, `require_auth()`
6. CRUD de frases + categorias (`dashboard.php`, `phrase_form.php`, `api/phrases.php`)
7. `player.js`: `speechSynthesis` com 3 velocidades e seleção de voz
8. `recorder.js`: `MediaRecorder` nativo + conversão final para WAV 16kHz mono via `OfflineAudioContext`, com preview e regravação
9. `api/analyze.php`: pipeline completo GPT-4o Audio + rate limit
10. `FeedbackCard` + regra de progressão + anti-frustração
11. `mastered.php` + `profile.php` + `delete_account`
12. `privacy.php`, `terms.php`, modal de consentimento + `api/consent.php` gravando `consented_at` — sem ele, o 403 do `analyze.php` trava todo usuário na primeira análise
13. Polish mobile: animações, toasts, confetti, testes no iPhone e Android
14. Deploy: upload via File Manager/FTP, `php.ini`, `.htaccess`, testar SSL

8.2 config.php (fora do public_html)

```
<?php
declare(strict_types=1);

define('DB_HOST', 'localhost');
define('DB_NAME', 'uXXXX_voxlyone');
define('DB_USER', 'uXXXX_voxly');
define('DB_PASS', '*****');

define('GOOGLE_CLIENT_ID', '....apps.googleusercontent.com');
define('GOOGLE_CLIENT_SECRET', '*****');
define('GOOGLE_REDIRECT_URI', 'https://voxly.hubfy.app/auth/callback.php');
```

```
define('OPENAI_API_KEY', 'sk-*****');
// Modelo preview: quando sair a versão estável, trocar apenas aqui
define('OPENAI_AUDIO_MODEL', 'gpt-4o-audio-preview');
// Teto global diário de análises (~$9/dia no pior caso; calibrar com o spike)
define('GLOBAL_DAILY_ANALYSES', 300);

define('APP_URL', 'https://voxly.hubfy.app');
define('LOG_FILE', __DIR__ . '/../logs/app.log');
```

8.3 .htaccess essencial

```
# Forçar HTTPS
RewriteEngine On
RewriteCond %{HTTPS} off
RewriteRule ^(.*)$ https://%{HTTP_HOST}/$1 [R=301,L]

# Headers de segurança
Header set X-Content-Type-Options "nosniff"
Header set X-Frame-Options "DENY"
Header set Referrer-Policy "strict-origin-when-cross-origin"

# Bloquear utilitários prefixados com _ (o _spike.php roda no Dia 1,
# ANTES deste .htaccess definitivo subir no deploy – e é deletado)
<FilesMatch "^_">
    Require all denied
</FilesMatch>
```

8.4 Configuração PHP no hPanel

max_execution_time	60
upload_max_filesize	8M
post_max_size	10M
memory_limit	256M
display_errors	Off (produção)

9. INSTRUÇÕES PARA O CLAUDE CODE

9. Instruções para o Claude Code

💡 Colar esta seção como contexto inicial (ou usar como CLAUDE.md na raiz do projeto) ao desenvolver no VS Code.

9.1 Contexto

Projeto VoxlyOne — app web de treino de pronúncia em inglês para brasileiros. Stack: PHP 8.2 puro (sem framework), MySQL (PDO), JS Vanilla, HTML/CSS mobile-first. Hospedagem compartilhada Hostinger. IA: GPT-4o Audio via cURL. Auth: Google OAuth 2.0 manual + sessões PHP.

9.2 Regras obrigatórias

- declare(strict_types=1) no topo de todo arquivo PHP
- TODA query via PDO prepared statements — proibido concatenar input em SQL
- TODA saída de dado de usuário via htmlspecialchars(ENT_QUOTES, 'UTF-8')
- TODO form POST com token CSRF validado server-side
- Secrets APENAS em config.php fora do public_html — nunca hardcoded
- Endpoints /api sempre retornam JSON { success, data|error } com http_response_code correto
- Toda página protegida começa com require_auth(); toda query filtra por user_id da sessão
- Nunca salvar o áudio em disco; processar e descartar
- Mensagens de erro ao usuário: genéricas e em PT-BR; detalhes técnicos só no log
- JS sem frameworks e sem build: arquivos estáticos em assets/js, ES6+, sem dependências externas — confetti da celebração em ~20 linhas de canvas próprio, sem CDN
- CSS mobile-first com variáveis CSS; breakpoints min-width

9.3 Checklist antes de cada commit

- Nenhum secret no código; prepared statements em tudo; XSS escapado
- Testado no celular (Chrome Android + Safari iOS) além do desktop
- Erros tratados com mensagem ao usuário + log no servidor
- php -l sem erros de sintaxe em todos os arquivos alterados

10. ESTIMATIVAS E ROADMAP

10. Estimativas e Roadmap

10.1 Cronograma MVP

Fase	Entregável	Tempo	Prioridade
1 — Base	OAuth Google + MySQL + estrutura	1–2 dias	Crítico
2 — CRUD	Frases e categorias completos	1–2 dias	Crítico
3 — Player	TTS com 3 velocidades	0.5 dia	Crítico
4 — Recorder	WAV no browser + preview	1–2 dias	Crítico
5 — IA	analyze.php + FeedbackCard	1–2 dias	Crítico
6 — Progressão	Regra do 8/10 + conquistas	1 dia	Crítico
7 — Legal	Privacy, consentimento, conta, terms, excluir	0.5 dia	Crítico
8 — Polish	Mobile, animações, testes reais	1–2 dias	Importante
TOTAL	—	7–12 dias	—

10.2 Custos

Hostinger compartilhada	Já contratada — \$0 adicional
Domínio + SSL	Já contratados — \$0
MySQL	Incluído no plano — \$0
Google OAuth	Gratuito
Web Speech + Web Audio	Gratuitos (browser)
OpenAI (GPT-4o Audio + mini)	~\$20–60/mês no MVP — saldo pré-pago com auto-recarga DESLIGADA como freio real de custo
TOTAL	~\$20–60/mês (somente OpenAI)

10.3 Roadmap de Evolução

Fase	Item	Gatilho
Fase 2	Migrar para VPS Hostinger (mesmos arquivos)	Lentidão ou limite de processos no compartilhado
Fase 2	Freemium: limite free + plano premium	Base de usuários validada
Fase 2	OpenAI TTS com cache MP3	Reclamações de voz robótica
Fase 3	Salvar áudios (evolução do aluno)	Pedido de usuários
Fase 3	Azure Pronunciation Assessment (fonema a fonema)	Diferenciação premium
Fase 3	Gamificação: streaks, ranking	Retenção

— Fim do documento VoxlyOne v2.5.0 —